

```
#####
# s243130.jl
# PlastOut A/S - Reactive GRASP + VND Local Search + ILS
#####

include("PlastOutReader.jl")
using Random

# =====
# Instance data
# =====
struct Instance
    name::String
    n::Int
    k::Int
    H::Int
    p::Vector{Int}
    rev::Vector{Int}
    pair::Matrix{Int}
    LB::Int
end

function load_instance(path::String)::Instance
    name, n32, LB32, rev32, pair32, k32, H32, p32 = read_instance(path)
    return Instance(
        String(name),
        Int(n32),
        Int(k32),
        Int(H32),
        Int.(p32),
        Int.(rev32),
        Array{Int}(pair32),
        Int(LB32)
    )
end

# =====
# Solution representation
# =====
mutable struct Solution
    assign::Vector{Int}           # 0 = unassigned, 1..k = line index
    line_items::Vector{Vector{Int}} # orders in each line
    pos_in_line::Vector{Int}      # position of i inside its line vector (0 if unassigned)
    time_used::Vector{Int}        # total time per line
    obj::Int                     # objective value
    sumPair::Matrix{Int}          # n x k : sumPair[i,l] = sum_{j in line l} pair[i,j]
end

function empty_solution(inst::Instance)::Solution
    return Solution(
        fill{0, inst.n},
        [Int[] for _ in 1:inst.k],
        fill{0, inst.n},
        fill{0, inst.k},
        0,
        zeros{Int, inst.n, inst.k}
    )
end

# =====
# Add / Remove
# =====
@inline function delta_add(inst::Instance, sol::Solution, i::Int, l::Int)::Int
    return inst.rev[i] + sol.sumPair[i, l]
end

function add!(inst::Instance, sol::Solution, i::Int, l::Int)
    @assert sol.assign[i] == 0
    @assert sol.time_used[l] + inst.p[i] <= inst.H

    sol.obj += delta_add(inst, sol, i, l)

    sol.assign[i] = l
    push!(sol.line_items[l], i)
    sol.pos_in_line[i] = length(sol.line_items[l])
    sol.time_used[l] += inst.p[i]

    @inbounds for x in 1:inst.n
        sol.sumPair[x, l] += inst.pair[x, i]
    end

    return nothing
end

function remove!(inst::Instance, sol::Solution, i::Int)
    l = sol.assign[i]
    @assert l != 0

    sol.obj -= (inst.rev[i] + sol.sumPair[i, l])

    @inbounds for x in 1:inst.n
        sol.sumPair[x, l] -= inst.pair[x, i]
    end

    sol.time_used[l] -= inst.p[i]

    vec = sol.line_items[l]
    idx = sol.pos_in_line[i]
    last_item = vec[end]
    vec[idx] = last_item
    sol.pos_in_line[last_item] = idx
    pop!(vec)
end

```

```

        sol.assign[i] = 0
        sol.pos_in_line[i] = 0

        return nothing
    end

# =====
# Debug / Validation
# =====
function compute_obj(inst::Instance, sol::Solution)::Int
    total = 0
    for l in 1:inst.k
        S = sol.line_items[l]

        @inbounds for i in S
            total += inst.rev[i]
        end

        m = length(S)
        @inbounds for a in 1:m-1
            i = S[a]
            for b in a+1:m
                j = S[b]
                total += inst.pair[i, j]
            end
        end
    end
    return total
end

function check_solution(inst::Instance, sol::Solution)::Bool
    for l in 1:inst.k
        @assert sol.time_used[l] <= inst.H
        tsum = 0
        for (pos, i) in pairs(sol.line_items[l])
            @assert sol.assign[i] == 1
            @assert sol.pos_in_line[i] == pos
            tsum += inst.p[i]
        end
        @assert tsum == sol.time_used[l]
    end
    @assert sol.obj == compute_obj(inst, sol)
    return true
end

# =====
# Neighborhoods: relocate + swap
# =====
@inline function can_move(inst::Instance, sol::Solution, i::Int, to::Int)::Bool
    from = sol.assign[i]
    @assert from != 0
    return from != to && (sol.time_used[to] + inst.p[i] <= inst.H)
end

@inline function delta_move(inst::Instance, sol::Solution, i::Int, to::Int)::Int
    from = sol.assign[i]
    return (inst.rev[i] + sol.sumPair[i, to]) - (inst.rev[i] + sol.sumPair[i, from])
end

function move!(inst::Instance, sol::Solution, i::Int, to::Int)
    @assert can_move(inst, sol, i, to)
    remove!(inst, sol, i)
    add!(inst, sol, i, to)
end

@inline function can_swap(inst::Instance, sol::Solution, i::Int, j::Int)::Bool
    li = sol.assign[i]
    lj = sol.assign[j]
    @assert li != 0 && lj != 0
    li == lj && return false

    new_t_li = sol.time_used[li] - inst.p[i] + inst.p[j]
    new_t_lj = sol.time_used[lj] - inst.p[j] + inst.p[i]

    return new_t_li <= inst.H && new_t_lj <= inst.H
end

@inline function delta_swap(inst::Instance, sol::Solution, i::Int, j::Int)::Int
    li = sol.assign[i]
    lj = sol.assign[j]

    gain_li = (inst.rev[j] + sol.sumPair[j, li] - inst.pair[j, i]) - (inst.rev[i] + sol.sumPair[i, li])
    gain_lj = (inst.rev[i] + sol.sumPair[i, lj] - inst.pair[i, j]) - (inst.rev[j] + sol.sumPair[j, lj])

    return gain_li + gain_lj
end

function swap!(inst::Instance, sol::Solution, i::Int, j::Int)
    @assert can_swap(inst, sol, i, j)

    li = sol.assign[i]
    lj = sol.assign[j]

    remove!(inst, sol, i)
    remove!(inst, sol, j)
    add!(inst, sol, i, lj)
    add!(inst, sol, j, li)

    return nothing
end

# =====
# Insert helper

```

```

# =====
@inline function best_insert_for_order(inst::Instance, sol::Solution, i::Int)
    best_l = 0
    best_gain = typemin(Int)

    for l in 1:inst.k
        if sol.time_used[l] + inst.p[i] <= inst.H
            g = inst.rev[i] + sol.sumPair[i, l]
            if g > best_gain
                best_gain = g
                best_l = l
            end
        end
    end

    return best_l, best_gain
end

# =====
# VND Local Search
# =====
function local_search_vnd!(inst::Instance, sol::Solution; rng::AbstractRNG=MersenneTwister(1))
    improved = true

    while improved
        improved = false

        # 1) RELOCATE
        orders = collect(1:inst.n)
        shuffle!(rng, orders)

        for i in orders
            li = sol.assign[i]
            li == 0 && continue

            for to in 1:inst.k
                to == li && continue

                if sol.time_used[to] + inst.p[i] <= inst.H
                    d = delta_move(inst, sol, i, to)
                    if d > 0
                        move!(inst, sol, i, to)
                        improved = true
                        break
                    end
                end
            end

            improved && break
        end

        improved && continue

        # 2) SWAP
        in_orders = [i for i in 1:inst.n if sol.assign[i] != 0]
        shuffle!(rng, in_orders)

        for a in 1:length(in_orders)-1
            i = in_orders[a]
            for b in a+1:length(in_orders)
                j = in_orders[b]
                sol.assign[i] == sol.assign[j] && continue

                if can_swap(inst, sol, i, j)
                    d = delta_swap(inst, sol, i, j)
                    if d > 0
                        swap!(inst, sol, i, j)
                        improved = true
                        break
                    end
                end
            end

            improved && break
        end

        improved && continue

        # 3) INSERT best unassigned
        unassigned = [i for i in 1:inst.n if sol.assign[i] == 0]
        shuffle!(rng, unassigned)

        best_i = 0
        best_l = 0
        best_gain = 0

        for i in unassigned
            l, g = best_insert_for_order(inst, sol, i)
            if l != 0 && g > best_gain
                best_gain = g
                best_i = i
                best_l = l
            end
        end

        if best_i != 0 && best_gain > 0
            add!(inst, sol, best_i, best_l)
            improved = true
        end
    end

    return sol
end

```

```

# =====
# Construction (RCL controlled by alpha)
# =====
struct Candidate
  i::Int
  l::Int
  gain::Int
  key::Float64
end

function greedy_randomized_construction(inst::Instance, alpha::Float64, rng::AbstractRNG)::Solution
  sol = empty_solution(inst)
  greedy_randomized_completion!(inst, sol, alpha, rng)
  return sol
end

function greedy_randomized_completion!(inst::Instance, sol::Solution, alpha::Float64, rng::AbstractRNG)
  orders = collect(1:inst.n)
  shuffle!(rng, orders)

  while true
    cand = Candidate{Int, Int, Int, Float64}[]
    sizehint!(cand, inst.n)

    for i in orders
      sol.assign[i] != 0 && continue

      best_l = 0
      best_gain = typemin{Int}()

      for l in 1:inst.k
        if sol.time_used[l] + inst.p[i] <= inst.H
          g = inst.rev[i] + sol.sumPair[i, l]
          if g > best_gain
            best_gain = g
            best_l = l
          end
        end
      end

      if best_l != 0 && best_gain > 0
        key = best_gain / max(inst.p[i], 1)
        push!(cand, Candidate(i, best_l, best_gain, key))
      end
    end

    isempty(cand) && break

    kmax = maximum(c.key for c in cand)
    kmin = minimum(c.key for c in cand)
    threshold = kmax - alpha * (kmax - kmin)

    rcl = Candidate{Int, Int, Int, Float64}[]
    for c in cand
      if c.key >= threshold
        push!(rcl, c)
      end
    end

    chosen = rand(rng, rcl)
    add!(inst, sol, chosen.i, chosen.l)
  end

  return sol
end

# =====
# ILS / Ruin & Recreate
# =====
@inline function order_strength(inst::Instance, sol::Solution, i::Int)::Int
  l = sol.assign[i]
  @assert l != 0
  return inst.rev[i] + sol.sumPair[i, l]
end

function perturb_ruin!(inst::Instance, sol::Solution, rng::AbstractRNG; r::Int=10, noise_frac::Float64=0.30)
  selected = [i for i in 1:inst.n if sol.assign[i] != 0]
  isempty(selected) && return Int[]

  strength = [(i, order_strength(inst, sol, i)) for i in selected]
  sort!(strength, by = x -> x[2]) # weakest first

  r = min(r, length(strength))
  pool_size = max(r, Int(ceil(r * (1.0 + noise_frac))))
  pool_size = min(pool_size, length(strength))

  pool = strength[1:pool_size]
  shuffle!(rng, pool)
  to_remove = pool[1:r]

  removed = Int{Int}[]
  sizehint!(removed, r)

  for (i, _) in to_remove
    remove!(inst, sol, i)
    push!(removed, i)
  end

  return removed
end

function ils_ruin_recreate(inst::Instance, start::Solution, alpha::Float64, rng::AbstractRNG;
  shakes::Int=6, r_frac::Float64=0.12, r_min::Int=5, r_max::Int=25)

```

```

current = deepcopy(start)
best = deepcopy(start)

for _ in 1:shakes
    selected_count = count(==(0), current.assign)
    selected_count == 0 && break

    r = Int(round(r_frac * selected_count))
    r = max(r, r_min)
    r = min(r, r_max)

    cand = deepcopy(current)

    perturb_ruin!(inst, cand, rng; r=r, noise_frac=0.30)
    greedy_randomized_completion!(inst, cand, alpha, rng)
    local_search_vnd!(inst, cand; rng=rng)

    if cand.obj > current.obj
        current = cand
        if current.obj > best.obj
            best = deepcopy(current)
        end
    end
end

return best
end

# =====
# Reactive alpha selection
# =====
mutable struct ReactiveAlpha
    alphas::Vector{Float64}
    p::Vector{Float64}
    A::Vector{Float64}
    count::Vector{Int}
    q::Vector{Float64}
end

function init_reactive(alphas::Vector{Float64})::ReactiveAlpha
    m = length(alphas)
    return ReactiveAlpha(
        alphas,
        fill(1.0 / m, m),
        zeros(Float64, m),
        zeros(Int, m),
        ones(Float64, m)
    )
end

function update_average(old_avg::Float64, old_count::Int, new_value::Float64)
    return (old_avg * old_count + new_value) / (old_count + 1)
end

function pick_alpha(r::ReactiveAlpha, rng::AbstractRNG)
    x = rand(rng)
    acc = 0.0
    for i in eachindex(r.p)
        acc += r.p[i]
        if x <= acc
            return i, r.alphas[i]
        end
    end
    return length(r.alphas), r.alphas[end]
end

function update_reactive!(r::ReactiveAlpha, aidx::Int, sol_revenue::Float64, best_revenue::Float64)
    r.count[aidx] += 1

    old_count = r.count[aidx] - 1
    r.A[aidx] = update_average(r.A[aidx], old_count, sol_revenue)

    for i in eachindex(r.alphas)
        if r.count[i] > 0
            r.q[i] = r.A[i] / max(best_revenue, 1.0)
        else
            r.q[i] = 1.0
        end
    end

    sum_q = sum(r.q)
    if sum_q <= 1e-12
        fill!(r.p, 1.0 / length(r.alphas))
        return nothing
    end

    for i in eachindex(r.p)
        r.p[i] = r.q[i] / sum_q
    end

    return nothing
end

function print_reactive_stats(r::ReactiveAlpha)
    println("\nReactive alpha stats:")
    for i in eachindex(r.alphas)
        println("  alpha=$(r.alphas[i])  p=$(round(r.p[i], digits=6))  count=$(r.count[i])  avg=$(round(r.A[i], digits=2))  q=$(round(r.q[i], digits=6))")
    end
end

# =====
# Logging

```

```

# =====
function log_header(io)
    println(io, "iter,elapsed_s,best_obj,curr_obj,alpha,selected_orders,gap_abs,gap_pct")
end

function log_row(io, iter::Int, elapsed::Float64, best_obj::Int, curr_obj::Int,
    alpha::Float64, selected::Int, LB::Int)
    gap_abs = LB - best_obj
    gap_pct = LB > 0 ? 100.0 * gap_abs / LB : 0.0
    println(io, "$(iter),$(round(elapsed,digits=3)),$(best_obj),$(curr_obj),$(alpha),$(selected),$(gap_abs),$(round(gap_pct,digits=3))")
end

function alpha_log_header(io, r::ReactiveAlpha)
    cols = ["iter","elapsed_s","best_obj","gap_abs","gap_pct"]
    for a in r.alphas
        push!(cols, "p_alpha_$(a)")
    end
    println(io, join(cols, ","))
end

function alpha_log_row(io, r::ReactiveAlpha, iter::Int, elapsed::Float64, best_obj::Int, LB::Int)
    gap_abs = LB - best_obj
    gap_pct = LB > 0 ? 100.0 * gap_abs / LB : 0.0

    row = String[]
    push!(row, string(iter))
    push!(row, string(round(elapsed, digits=3)))
    push!(row, string(best_obj))
    push!(row, string(gap_abs))
    push!(row, string(round(gap_pct, digits=3)))
    for pi in r.p
        push!(row, string(round(pi, digits=6)))
    end
    println(io, join(row, ","))
end

# =====
# Reactive GRASP main loop
# =====
function reactive_grasp(inst::Instance, timelimit_s::Float64=10.0, seed::Int=1,
    logpath::Union{Nothing,String}=nothing,
    alpha_logpath::Union{Nothing,String}=nothing,
    shakes::Int=6, r_frac::Float64=0.12)

    rng = MersenneTwister(seed)
    r = init_reactive([0.05, 0.10, 0.20, 0.35, 0.50])

    best_sol = empty_solution(inst)
    best_sol.obj = typemin(Int)

    t0 = time()
    iter = 0

    io = nothing
    if logpath !== nothing
        io = open(logpath, "w")
        log_header(io)
    end

    aio = nothing
    if alpha_logpath !== nothing
        aio = open(alpha_logpath, "w")
        alpha_log_header(aio, r)
    end

    while (time() - t0) < timelimit_s
        iter += 1

        aidx, alpha = pick_alpha(r, rng)

        sol = greedy_randomized_construction(inst, alpha, rng)
        local_search_vnd!(inst, sol; rng=rng)
        sol = ils_ruin_recreate(inst, sol, alpha, rng; shakes=shakes, r_frac=r_frac, r_min=5, r_max=25)

        if sol.obj > best_sol.obj
            best_sol = sol
        end

        update_reactive!(r, aidx, Float64(sol.obj), Float64(best_sol.obj))

        if aio !== nothing
            elapsed = time() - t0
            alpha_log_row(aio, r, iter, elapsed, best_sol.obj, inst.LB)
        end

        if io !== nothing
            elapsed = time() - t0
            selected = count(==(0), sol.assign)
            log_row(io, iter, elapsed, best_sol.obj, sol.obj, alpha, selected, inst.LB)
        end
    end

    io !== nothing && close(io)
    aio !== nothing && close(aio)

    return best_sol, r
end

# =====
# Output writer (.sol)
# =====
function write_solution(path::String, inst::Instance, sol::Solution)
    open(path, "w") do io

```

```

        for l in 1:inst.k
            vec = sol.line_items[l]
            if !isempty(vec)
                for (idx, i) in pairs(vec)
                    if idx == 1
                        print(io, i)
                    else
                        print(io, " ", i)
                    end
                end
            end
            end
            print(io, "\n")
        end
    end
end

# =====
# Main
# =====
function main(args)
    if length(args) < 3
        println("Usage: julia s243130.jl <instance_path> <solution_path> <timelimit_s>")
        return
    end

    instance_path = abspath(args[1])
    solution_path = abspath(args[2])
    timelimit_s = parse(Float64, args[3])

    println("Working dir: ", pwd())
    println("Instance path: ", instance_path)
    println("Solution path: ", solution_path)

    inst = load_instance(instance_path)

    outdir = dirname(solution_path)
    mkpath(outdir)

    logpath = abspath(outdir, "log.csv")
    alpha_logpath = abspath(outdir, "alpha_log.csv")

    shakes = 6
    r_frac = 0.12

    best, stats = reactive_grasp(inst;
                                timelimit_s=timelimit_s,
                                seed=1,
                                logpath=logpath,
                                alpha_logpath=alpha_logpath,
                                shakes=shakes,
                                r_frac=r_frac)

    gap_abs = inst.LB - best.obj
    gap_pct = inst.LB > 0 ? 100.0 * gap_abs / inst.LB : 0.0

    println("Instance: ", inst.name)
    println("LB: ", inst.LB, " Best found: ", best.obj)
    println("Gap abs: ", gap_abs, " Gap %: ", round(gap_pct, digits=2), "%")
    println("Log: ", logpath)
    println("Alpha log: ", alpha_logpath)

    print_reactive_stats(stats)

    write_solution(solution_path, inst, best)
end

if abspath(PROGRAM_FILE) == @__FILE__
    main(ARGS)
end

```